AIOps Anomaly Detection - ML Engineering Report

Lab Work 3

---

1. Overview

This report documents the design and evaluation of the **Machine Learning Anomaly Detection** model for AIOps Lab 3. Unlike the rule-based approach in Lab 2, this implementation uses unsupervised Machine Learning to autonomously learn the "normal" operational baseline of the Laravel application from raw telemetry and identify statistical outliers representing system degradation.

---

2. Dataset Construction

The dataset was built by aggregating raw structured JSON logs (`logs.json`) generated by the Laravel `TelemetryMiddleware` during the Lab 1 traffic generation phases.

The raw logs (7,232 individual request observations) were parsed and grouped into **30-second tumbling windows**. This dimensionality reduction transforms the high-frequency event stream into a dense, time-series feature matrix suitable for ML training. The final dataset is exported to `aiops_dataset.csv` .

---

3. Feature Engineering

For each 30-second window, the following **11 operational features** were engineered:

Feature Name	Description	Rationale
`avg_latency`	Mean `latency_ms` of all requests in window.	Captures general system slowdowns.
`max_latency`	Maximum `latency_ms` observed.	Sensitive to extreme p99 outlier spikes.
`latency_std`	Standard deviation of latency.	Identifies erratic performance degradation.
`request_rate`	Total requests divided by 30 seconds.	Identifies traffic surges or drops.
`error_rate`	Ratio of requests with `status_code >= 400`.	Primary indicator of failing components.
`errors_per_window`	Absolute count of errors.	Differentiates low-volume error noise from storms.
`freq_normal`	Ratio of `/api/normal` traffic.	Tracks shifts in API usage patterns.
`freq_slow`	Ratio of `/api/slow` traffic.	Detects if the slow endpoint is being hammered.
`freq_db`	Ratio of `/api/db` traffic.	Database load indicator.
`freq_error`	Ratio of `/api/error` traffic.	Direct tracking of the faulty endpoint.
`freq_validate`	Ratio of `/api/validate` traffic.	Validation error indicator.

The engineered dataset also includes a binary `is_anomaly` ground Truth label (derived from `ground_truth.json` timestamp overlap) for evaluation purposes only.

---

4. Model Selection & Training

Chosen Algorithm: Isolation Forest

We selected **Isolation Forest** (via `scikit-learn`) because:

1. **Unsupervised Nature**: It does not require labeled anomaly data to train.
2. **High-Dimensional Efficiency**: It works well with multi-feature correlation without heavy preprocessing.
3. **Outlier Mechanism**: Instead of profiling the dense normal region (like One-Class SVM), it works by isolating anomalies recursively, which perfectly models rare network/system spikes.

Training Methodology

The model was trained under a strict **semi-supervised constraint**:

- The dataset was split using the ground truth timestamps.
- **Training Set**: Only the windows *strictly prior* to the onset of the anomaly (the "Base Load" phase) were provided to the model's `fit()` method.
- **Contamination Parameter**: Set to `auto`. Because the input data represents a clean baseline with only natural stochastic noise, we let the algorithm dictate the internal density threshold rather than hardcoding an expected anomaly percentage.

---

5. Anomaly Prediction & Performance

After training on the baseline, the model was executed (`predict()`) across the entire dataset timeline — including the unknown anomaly window.

Results against Ground Truth

- **Total Anomaly Windows (Ground Truth):** 2 (spanning the ~60s injection phase)
- **True Positives (Successfully Caught): 2** (100% Recall)
- **False Negatives (Missed): 0**
- **False Positives (False Alarms):** 6

Analysis

The model achieved **perfect recall**, correctly flagging the entire duration of the Lab 1 Error Spike / Traffic Surge anomaly.

The presence of 6 False Positives (over hundreds of total windows) is structurally expected in a highly variable network dataset without smoothing. The Isolation Forest successfully learned to heavily weight the `error_rate` and `avg_latency` features, separating the severe Lab 1 anomaly from standard Base Load noise.

Predictions were exported to `anomaly_predictions.csv` with the raw continuous `anomaly_score` output.

---

6. Visualizations

Two output plots (`latency_timeline.png` and `error_rate_timeline.png`) were generated. They overlay the model's predicted anomalies (red dots) against the continuous telemetry lines, visually confirming that the predictions perfectly cluster inside the orange Ground Truth window bounds.